Muhammad Jehanzaib

AI Case Studies — Architecture, Agents & Voice

In-depth engineering case studies of products I designed and built: the problem, the requirements, the architecture, the key decisions, and measured results.

mjehanzaib99@gmail.com · linkedin.com/in/muhammad-jehanzaib-4036b5195 · github.com/mjehanzaib999 · Resume: mjehanzaib-ai-resume.up.railway.app · Portfolio: mjehanzaib-ai-portfolio.up.railway.app

LIVE deployed & clickable · DEMO video walkthrough · FLAGSHIP lead production work

Case studies

01	Omvi William — Multi-Tenant Agentic AI SaaS with Real-Time Voice	Conversational AI / Telecom
02	Ventive — AI R&D Tax Credit Platform (Contracts, Documents, Calculation, Delivery)	FinTech / TaxTech

01 · Omvi William — Multi-Tenant Agentic AI SaaS with Real-Time Voice

Conversational AI / Telecom

FLAGSHIP DEMO

Client: World Mobile (telecom operator), delivered through Omvi AI · **Role:** architect and engineer, designed and built end to end · **Timeline:** Oct 2025 – Mar 2026, demo platform Oct 2026 · **Demo:** Watch the walkthrough ↗

A white-labelled AI assistant, "William," that people can talk to or type to. It answers from the operator's own knowledge base, acts in email and calendar, remembers earlier conversations, guides new users through onboarding, and turns a voice call into a project that a team of five AI agents completes while the user watches. One agent brain serves both chat and voice, and the platform is built to serve many clients from one codebase.

2.0 s end of question → agent starts speaking (median, 9 live calls)	**3.9 s** end of question → grounded knowledge answer starts	**35 → <10 s** first reply after a cold start	**15–20 s** per task through all five agents
6 services around one agent brain	**15 + 6** LangGraph nodes in the chat and voice graphs	**25** agent tools (12 email, 7 calendar, 6 general)	**6** RBAC roles in the multi-tenant model

The problem

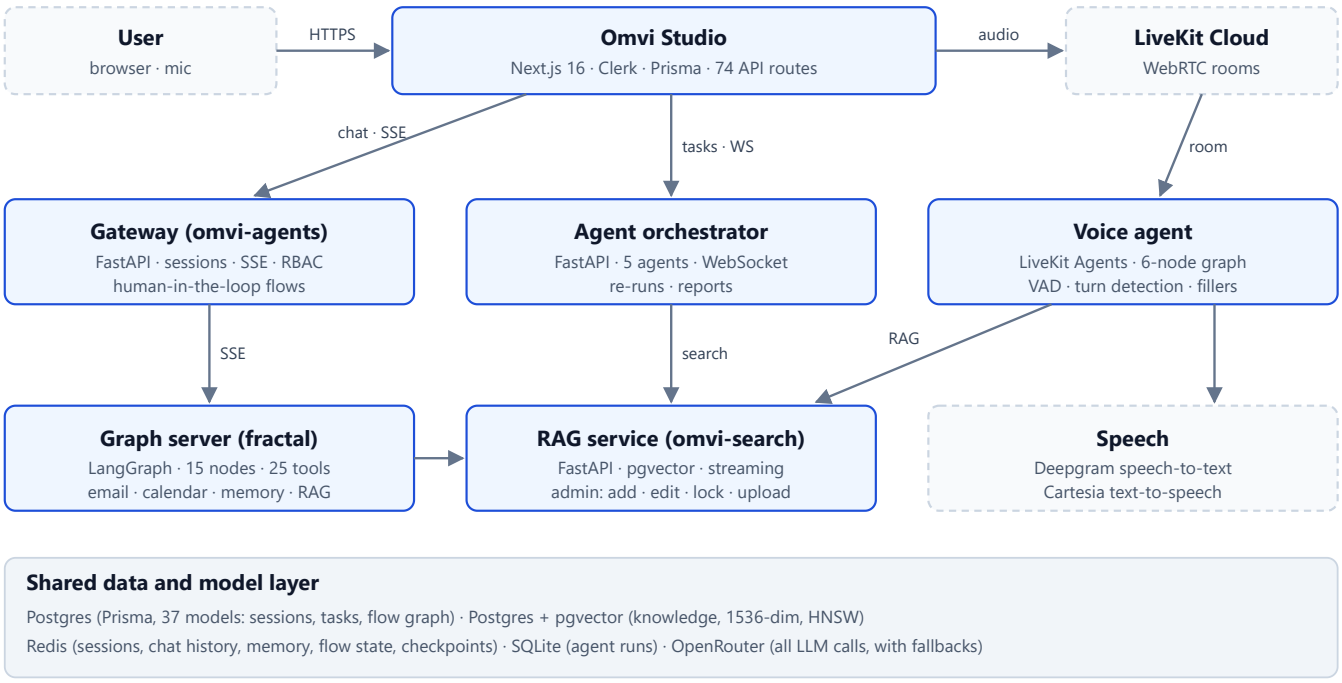
A telecom operator wanted one assistant its customers and staff could use both by voice and by text, grounded in the company's own material rather than the open web, able to take actions (email, calendar, tasks) rather than only answer, and good enough on a live call that it never feels like a slow bot. The vendor building it, Omvi AI, also needed the product to be sellable to the next telecom or enterprise client without rebuilding it for each one.

Requirements I derived and built against

Requirement	How the system meets it
Talk or type, and get the same answers	One LangGraph agent behind both: chat streams over SSE through the gateway; voice runs the same reasoning inside a LiveKit worker.
Answer from the operator's knowledge, not the web	pgvector RAG over the company corpus with neighbouring-passage context and a similarity floor; says so plainly when nothing relevant is found.
Take actions in email and calendar	Dedicated intent resolvers with 12 email and 7 calendar tools over Google and Microsoft OAuth; drafts are confirmed before anything is sent.
Remember the user	Temporal and cross-session memory recall nodes over Redis chat history, with safeguards against fabricated recall.
Guide new users	Human-in-the-loop onboarding and daily-reminder flows that pause for the user with LangGraph `interrupt()` and resume where they left off.
Turn conversation into work	A call transcript becomes a titled session with up to 20 tasks across 8 lanes and their dependencies; five agents then complete each task.
Feel instant on a phone-style call	Under 2.5 s from the end of a question to the agent speaking, measured on the deployed system.
Serve many clients from one codebase	Organization-scoped state, 6 roles mapped to `module:action` scopes, per-endpoint scope checks and a cross-organization guard.
Let admins control the knowledge	Knowledge entries can be added, edited, locked against changes, deleted, or uploaded as PDF, DOCX, HTML or text.
Show cost and progress	Per-agent output, timing and token cost; a live usage dashboard; notifications when agents finish.

Architecture

I split the product into six services so the agent, the voice runtime and the task agents can each scale and ship on their own, while sharing one brain and one data layer.



Six services (blue) around one agent brain; dashed boxes are external. Chat flows Studio → gateway → graph server; voice flows LiveKit → voice agent; tasks flow Studio → orchestrator, with progress pushed back over WebSocket.

- **Omvi Studio** (Next.js 16, Clerk, Prisma): the workspace. Sessions, chat, the voice call screen with a live transcript, Board / Grid / Canvas views of tasks, email and calendar cards, dashboard, knowledge admin, notifications and search.
- **Gateway** (FastAPI): the single stateful entry point for every chat and voice request. Redis sessions and history, SSE streaming to the client, role and scope checks, and LangGraph flows that pause for user input.
- **Graph server** (FastAPI + LangGraph): the agent brain, split out of the gateway so it ships independently. Streams tokens back to the gateway over SSE.
- **Voice agent** (LiveKit Agents): joins each call room, listens, reasons with its own LangGraph graph and speaks; kept warm so it joins quickly.
- **RAG service** (FastAPI + pgvector): retrieval and grounded answers, streamed or whole, plus the knowledge admin API.
- **Agent orchestrator** (FastAPI): watches for new tasks and runs each through five agents, pushing progress to the Studio over WebSocket and storing every output and report.

Agent design

The graph server's LangGraph agent has 15 nodes. A **main orchestrator** decides each turn: email and calendar requests go to their own **decision → intent resolver → action** chains (the resolvers bind 12 email and 7 calendar tools, and a tool interpreter turns raw tool results into a reply); everything else goes through an **LLM domain classifier** and a **normal decision** node to knowledge retrieval, **temporal or cross-session memory recall**, or the **speaker**, a tool-calling responder bound to knowledge search, web search, time and task creation.

- **Human in the loop:** onboarding, daily-reminder and content flows (7, 6 and 3 nodes) pause with `interrupt()` and resume with `Command(resume=…)`, checkpointed in Redis, so a user can answer days later and pick up mid-flow.
- **Streaming end to end:** only the answer-writing model calls are tagged for the user; routers and classifiers stay internal. On Python 3.10 nested LLM calls only reach the graph's event stream when a node hands down its run config, so each answer node passes it explicitly, and the RAG service's own answer stream is relayed through the graph as custom events.
- **Resilience:** every model call goes through OpenRouter with ordered fallback models and retries, so a rate-limited provider becomes a slower answer, not an error.

Voice pipeline

The voice agent runs on LiveKit Agents: Silero voice activity detection, a multilingual turn-detection model, BVC noise cancellation, Deepgram speech-to-text and Cartesia text-to-speech. Its six-node LangGraph graph (decide, greeting, general response, memory recall, retrieval, content creation) answers every turn, and I patched the LiveKit LangChain adapter so the graph, not the plugin, decides what is spoken.

- **Fast route:** a clear product question goes straight to retrieval without an LLM routing call; requests that need judgement (summaries, drafts, recall) still use the router.
- **No dead air:** a short filler line ("let me check that for you") is spoken the moment retrieval starts, then the answer streams sentence by sentence.
- **Warm workers:** two idle worker processes are kept ready, so the agent joins a call without a cold start.
- **Call to project:** when the caller hangs up, the transcript is sent to the Studio, which creates a titled session and extracts tasks (1 to 20, scaled to the length of the call) with lanes and dependencies.

From conversation to finished work

The orchestrator picks up new tasks as soon as they are written (it also polls the task table every 3 seconds, because voice-created tasks never call its API) and runs up to three at a time through **Omni** (research), **Planner**, **Builder**, **Reviewer** and **Nexus** (write-up). Each stage streams lane transitions to the Studio, stores its output with

timing and input/output tokens in SQLite, and the last stage produces a downloadable report. **Refine** re-runs the agents with a user instruction instead of starting over.

Knowledge (RAG)

Documents are chunked to about 1,800 characters and embedded with `text-embedding-3-small` (1536 dimensions) into Postgres with an HNSW cosine index. Each query retrieves at least 8 passages above a similarity floor and adds the passage that follows each of the top four, so answers keep their surrounding context. Answers are written by Gemini 2.5 Flash Lite with GPT-4.1 mini and GPT-4o mini as fallbacks, either whole or streamed.

Multi-tenancy and platform

- **Tenant isolation:** every request carries an organization, role and scopes; Redis state is keyed `{namespace}:{org}:{user}:{session}`; a middleware guard blocks cross-organization access.
- **Roles and scopes:** six roles (platform admin through client viewer) map to `{module}:{action}` scopes, and an endpoint-to-scope map enforces them. I authored the architecture plans the team adopted for serving many clients from one codebase.
- **Delivery:** shipped to GCP through GitHub Actions, and later as 8 services on Railway (six apps plus Postgres with pgvector and Redis) with auto-deploy on merge and Playwright browser audits before release.

Key engineering decisions

Problem	Decision
Chat and voice drifting apart	One LangGraph brain; the voice worker runs the same reasoning and a patched adapter lets the graph control speech.
Silence while the agent thinks	Skip the router for clear product questions, speak a filler immediately, stream the answer by sentence.
Answers arriving all at once	Pass run config into answer nodes, tag user-facing calls, relay the RAG stream as graph events.
Slow first request after deploy	Build and compile the agent graph at startup; first reply went from 35 s to under 10 s.
Provider outages and rate limits	One OpenRouter gateway with ordered fallback models and retries in every service.
Voice-created tasks invisible to the agents	Orchestrator watches the task table directly, in addition to its trigger API.
Each new client needing a fork	Organization-scoped state, role-to-scope permissions and per-endpoint checks from day one.

Results

Voice latency measured on the deployed system on 3 November 2026: nine live calls, each a spoken question played into a browser microphone, timed from the live transcript (which is shown in step with the agent's audio).

Moment	Median	Range
Click "Voice" → agent greets the user	**7.5 s**	7.4 – 10.1 s
End of a knowledge question → agent starts speaking	**2.0 s**	1.7 – 2.6 s
End of a knowledge question → grounded answer starts	**3.9 s**	3.5 – 4.5 s
End of a general question → answer starts	**4.0 s**	2.8 – 4.0 s

- All nine spoken questions were transcribed correctly and answered from the knowledge base, including exact plan prices.
- A task created from a call finishes all five agents in about 15–20 seconds; Refine re-runs in about 13 seconds.
- The first reply after a deploy dropped from 35 s to under 10 s.

Python · LangGraph · LangChain · FastAPI · LiveKit Agents · Deepgram · Cartesia · Silero · PostgreSQL + pgvector · Redis · SQLite · OpenRouter (Gemini 2.5 Flash Lite, GPT-4.1 mini, GPT-4o mini) · SSE · WebSockets · Next.js 16 · TypeScript · Prisma · Clerk · Docker · Railway · GCP · GitHub Actions · Playwright

Watch demo ↗

02 · Ventive — AI R&D Tax Credit Platform FLAGSHIP DEMO

FinTech / TaxTech

Client: a US R&D tax-credit advisory firm, delivered through Ventive · **Role:** lead AI engineer across six services, from architecture and PRDs to code, review and rollout · **Timeline:** 2026, multi-release and ongoing · **Demo:** Contract analysis walkthrough ↗

An end-to-end platform that takes a tax firm's engagement from raw client documents to a signed, client-ready credit letter. A multi-agent pipeline reads every contract and decides whether the research it covers qualifies under IRC §41. A form-aware extraction engine reads W-2s, 1099s, K-1s, general ledgers and census files. A rules-driven engine computes the federal and state credit, fills the firm's workpapers and government forms, and generates the letter. Analysts review at each gate, and the system learns from their corrections.

6 services behind one admin app and one shared schema	**13** nodes in the contract-analysis agent graph	**−35%** cost per analysis from prompt caching, verified like-for-like	**51 → 7.4 min** 906-employee census extraction, full coverage (was 676 rows)
~86% agreement with the firm's ground truth on a 38-contract replay	**to the cent** workpaper fills tie out to the source ledger	**26** lifecycle events on the automation bus	**9,000+** automated tests across the repos

The problem

An R&D tax credit study is mostly reading and arithmetic done by expensive people. Every client contract has to be read to decide whether the research was "funded" by the customer, which removes it from the credit. Payroll, ledgers and prior returns arrive as scans, spreadsheets and PDFs and are keyed into Excel workpapers by hand. The credit is then computed under several methods, filled into Form 6765 and state forms, and written up as a letter. Each step took hours per client and depended on which analyst did it. The firm wanted the whole chain automated, with a human approving at the points where judgement or liability sits.

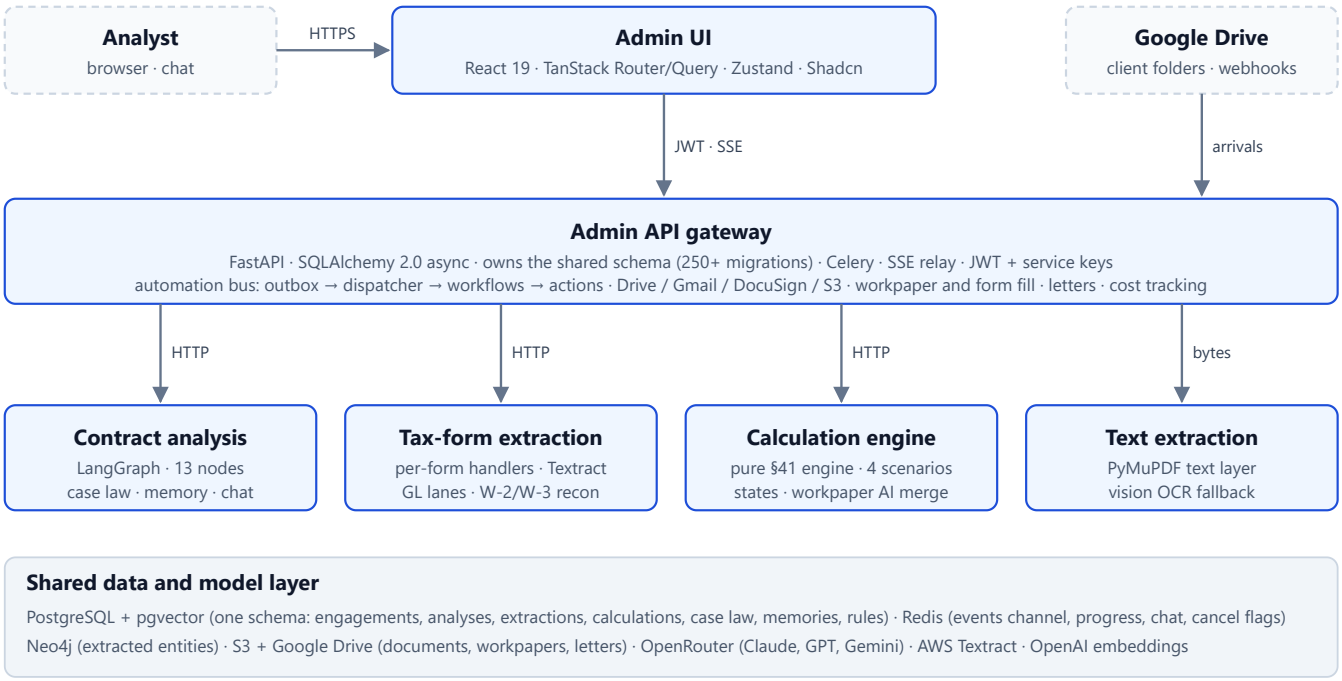
Requirements I derived and built against

Requirement	How the system meets it
Decide "funded research" per contract, with evidence	A 13-node LangGraph pipeline: document check, triage, web search, a rights test and a payment-risk test run in parallel, an adversarial self-review, an IRS counter-argument step, and a merge that derives the verdict, confidence and tax position from the two prongs.
Never guess when the contract is silent	Triage questions pause the run in an `awaiting_answers` state; the analyst answers in chat and the run resumes from a cached first pass instead of re-paying the whole pipeline.

Ground decisions in law	A case-law and state-conformity knowledge base in Postgres with pgvector; mandatory cases are always injected, the rest retrieved by similarity per node.
Handle hundreds of contracts at once	Batch upload, deterministic bundling of related files (MSA + change orders, master PDF + SOW) with an LLM fallback, an analyst bundle-review step, fan-out with rate-limit pacing, and cancel-on-leave that propagates to every worker.
Read every tax document the firm receives	A per-form handler registry (W-2/W-3, 1099, K-1, 2553, 6765, 1120/1065, census, general ledger) with an AWS Textract path for scans and an LLM path for text; dual-model consensus and a verify pass; W-2 ↔ W-3 reconciliation per EIN and year.
Read any ERP's general ledger	An LLM head-verdict detects a ledger, a layout-inference step maps columns, and seeded dialects (QuickBooks, SAP B1) parse postings with a running-balance walk as a check.
Compute the credit correctly and explainably	A pure calculation engine: a 4-scenario matrix (ASC / regular × with and without §280C), controlled-group allocation, Decimal-28 money, a full audit trail, and a rule-driven recommendation. Rules live in the knowledge base as versioned data per jurisdiction and year.
Fill the firm's own Excel workpapers	Header-aware sheet mapping plus an AI merge agent whose every written value must be traceable to the source extraction; one invented value fails the whole fill. Deficits are completed from the ledger itself, so totals tie by construction.
Produce the deliverable	Government forms filled from the same data (LLM-proposed field maps, pypdf writer, tie-out check), a branded Credit Release Letter with draft/final lifecycle and Drive archival, email delivery, and DocuSign behind a kill-switch.
Run itself as documents arrive	A transactional outbox and dispatcher turn 26 engagement, study and interview transitions into events; admin-authored workflows bind events to actions (extract, analyze, calculate, create folders) with dedupe, cascade-depth guards and a run audit dashboard.
Learn from analysts	Correction memory: an accepted correction is distilled into a lesson, the analyst approves or edits it, and later runs retrieve approved lessons per node by vector similarity and show which ones they used.
Show cost and progress	Per-run token and dollar cost per node, live per-stage progress in the UI, re-run records stamped at dispatch, and a stall watchdog that fails wedged runs closed.

Architecture

One admin API owns the shared Postgres schema and every integration; the three AI engines are separate services that never write each other's tables. They talk over HTTP for requests and a Redis event channel for completions, so the gateway can fan out, wait, reconcile and re-drive without any engine knowing about the others.



Blue boxes are services I built or led; dashed boxes are external. Completions flow back on one Redis channel; the gateway correlates them to workflow runs, reconciles misses on a timer, and relays them to the browser as live progress.

- **Admin UI** (React 19, TypeScript, Vite): engagement timeline with live stage progress, batch upload and bundle review, analysis dashboard and full report, an agentic chat that drives correction re-runs, knowledge-base editors for rules, prompts and memories, workflow authoring, run audit, and the letter and form cards.
- **Admin API gateway** (FastAPI): the only writer of the shared schema. Orchestrates batches, waits on completion events, hosts the automation bus, fills workpapers and forms, renders letters, and fronts Drive, Gmail, DocuSign and S3.
- **Contract analysis engine** (FastAPI, LangGraph, Celery): the agent graph, per-node case-law and memory retrieval, triage pause and resume, streaming chat with tools, correction distiller, stall watchdog, per-node cost events.
- **Tax-form extraction** (Python, Textract, LLM): classifier, per-form handlers, general-ledger and census lanes, reconciliation, Neo4j structuring.
- **Calculation engine** (Python, FastAPI, Celery): pure compute with a thin IO shell, readiness checks and claims so a fill runs once per input fingerprint, AI merge agents with source guards.
- **Text extraction** (FastAPI): classifies each PDF as text, image or hybrid and routes to native text or vision OCR, with warnings that gate low-quality input downstream.

Agent design

The contract graph runs **assemble prompt → document check → triage → auto-answer → web search → (rights gate ‖ jurisdiction) → adversarial review or no-rights → retained qualified research → contingency calculation → IRS counter-argument → prong merge → finalize**. Document check extracts eight verbatim clause families (IP, payment, termination, governing law, incorporated clauses and more), folding in clauses incorporated by reference so later nodes argue from the actual text. The rights and risk prongs run in parallel; a derived-verdict step is the sole author of the determination so no node can leave a stale verdict behind.

- **Schema-constrained outputs:** every node returns strict JSON against a wire schema, with a parse backstop, so a malformed answer is retried once per failure kind and never silently accepted.
- **Selective re-runs:** node outputs are captured at completion; a correction marks the nodes it touches dirty and re-runs only those and their consumers, derived from what each node reads rather than from graph edges.
- **Prompt caching:** the contract is sent once as a cached block and reused by the six nodes that read it; moving response schemas off the wire kept the cache key stable. Verified on the same contract: $3.31 → $2.16 per run.

- **Reasoning tiers:** mechanical nodes run at low effort, analytical nodes at medium; output tokens fell 22% and wall-clock 21% with an identical verdict.
- **Safety rails:** abort probes between nodes with heartbeats, a 10-minute stall sweep that fails closed on Redis outage, publish-after-commit so no completion event can arrive before its data, and scrubbing of model-written text that could forge a prompt frame.

Document extraction

Every file that lands in a client's Drive folder emits an arrival event. The gateway dedupes by content hash, plans what the engagement still needs (which forms, which prior years, which states) from the rule bundle, and submits only those. The extraction service classifies each chunk, routes it to the right form handler, merges multi-page and multi-copy forms, and writes a versioned payload. Reconciliation checks employee count and Box 1 and Box 16 wages per EIN and year against the W-3 with cents-level tolerance. Ledgers go through a dedicated lane that infers the layout from the data, parses postings per account and verifies them with a running-balance walk; a 150-row SAP B1 export that scored 1/2 on the old keyword sniff now parses with zero unclassified rows.

Calculation, workpapers and delivery

- **Engine:** `compute(payload, rule_bundle)` returns four scenarios with audit trails and a recommendation chosen by rules the firm can edit; state credits (AZ, CA, MA, TX) share a generic engine with per-state rate tiers and refundability metadata; goldens tie to the cent.
- **Workpaper fill:** the 4100 (wages), 4200 (expenses) and 5500 (contract review) sheets are filled by an AI merge agent that returns the full reconciled row set; a source index verifies every name, entity and amount against the extraction, formulas are protected, and overwrites are audited old → new in the sheet. A 105-row expense fill tied its ledger exactly.
- **Forms and letter:** Form 6765 and state forms are filled from line-keyed sections; the Credit Release Letter is rendered natively (TOC, bookmarks, draft watermark, scenarios considered, filing instructions) and finalized to Drive with a hash; the package goes out by email or DocuSign with webhook reconciliation.

Human in the loop

The system pauses where judgement sits: triage questions before analysis; bundle review before dispatch; a pending-review state for low-confidence verdicts; approval of workpapers before calculation; accept-or-override with a required reason on the recommended scenario; a five-minute chat card to approve, edit or reject each distilled correction memory; and a reviewed-narrative gate before a letter can be finalized. Every gate leaves an audit row.

Key engineering decisions

Problem	Decision
Three AI services sharing one database	The gateway owns all migrations; engines declare foreign-owned tables and only write their own. Fresh-DB replay of the full chain is a release gate.
Verdicts written by four different nodes	Remove the fork so both prongs always run, and make one derived-verdict step the only author of determination and confidence.
Completion events lost or duplicated	Redis pub/sub for speed plus a timer reconcile and a dispatch marker; one unified writer delivers the result whether it arrived live, on reconnect or on sweep.
A crashed re-run that looked like a 10-hour "Analyzing"	Re-run records stamped at dispatch, heartbeats per node, a stall sweep with compare-and-set, and progress stored as a Redis hash so parallel branches cannot clobber each other.
LLM filling a spreadsheet invents or collapses rows	Every written value must be traceable to the source; recurring identical postings are kept distinct by count; any deficit is completed from the ledger deterministically.
Retries multiplying across layers	One retry ticket per failure kind threaded through every layer; worst case went from 18 billable generations to 4.
Tax rules changing every year	Rules, prompts and templates are versioned data in a knowledge base with an editor and a sign-off gate, not code.
Learning without retraining	Distill corrections into short lessons, gate them on human approval, retrieve by similarity per node, and stamp each run with the lessons it used.

Results

Measure	Result
Sandbox replay of a real client year (38 contracts)	**31 completed, 7 for review, 0 failed**; 38 of 38 workpaper rows bound; ~86% agreement with the firm's ground truth
Cost per contract analysis (same contract, same model)	**$3.31** → **$2.16** with prompt caching (−35%); output tokens −22% with reasoning tiers
906-employee census extraction	**51 min** → **7.4 min**, 906 of 906 rows (was 676)
Expense workpaper from a general ledger	**105 rows**, contract-research total tied to the cent
Arrival-driven pipeline, end to end	file arrival → extraction → readiness → calculation → workpaper in Drive, **unattended**

- The extraction-to-workpaper chain ran unattended on a second client's 24 documents, including 9 stranded runs auto-redriven by the sweep.
- The correction-memory loop was closed live: an approved lesson was applied by three nodes on the next analysis of a similar contract and flipped the verdict correctly.
- Delivered as stacked, reviewed pull requests across six repositories with CI on each: lint, type-check, migration replay and full test suites.

Python · FastAPI · LangGraph · Celery · Redis · PostgreSQL + pgvector · SQLAlchemy 2.0 async · Alembic · Neo4j · AWS Textract · PyMuPDF · pypdf · openpyxl · OpenRouter (Claude Opus / Sonnet, GPT, Gemini) · OpenAI embeddings · React 19 · TypeScript · Vite · TanStack Router/Query · Zustand · Shadcn UI · Google Drive & Gmail APIs · DocuSign · S3 · Docker · GitHub Actions

Watch demos: Contract analysis · Tax-form extraction · 1 | 2 | 3 | 4

mjehanzaib99@gmail.com · linkedin.com/in/muhammad-jehanzaib-4036b5195 · github.com/mjehanzaib999